Tarea API's
Nicolás Alejandro Diosa Benavides
Ingeniería de Software II
Docente: Sergio Vargas Pedraza

1. Elegir UNA API de la lista

2. En Postman crear mínimo 5 requests:
   - GET todos los recursos
   - GET uno por ID o filtro
   - Si tiene POST → crear uno
   - Usar query params en al menos una request
   - Escribir mínimo 2 tests automáticos

3. Exportar la colección como .json

4. Documento con capturas y responder:
   → ¿Qué API elegiste y por qué?
   → ¿Qué datos devuelve?
   → ¿Usa token o no? ¿Qué tipo?
   → ¿Qué código de estado recibiste en cada request?
   → ¿Qué aprendiste diferente a JSONPlaceholder?

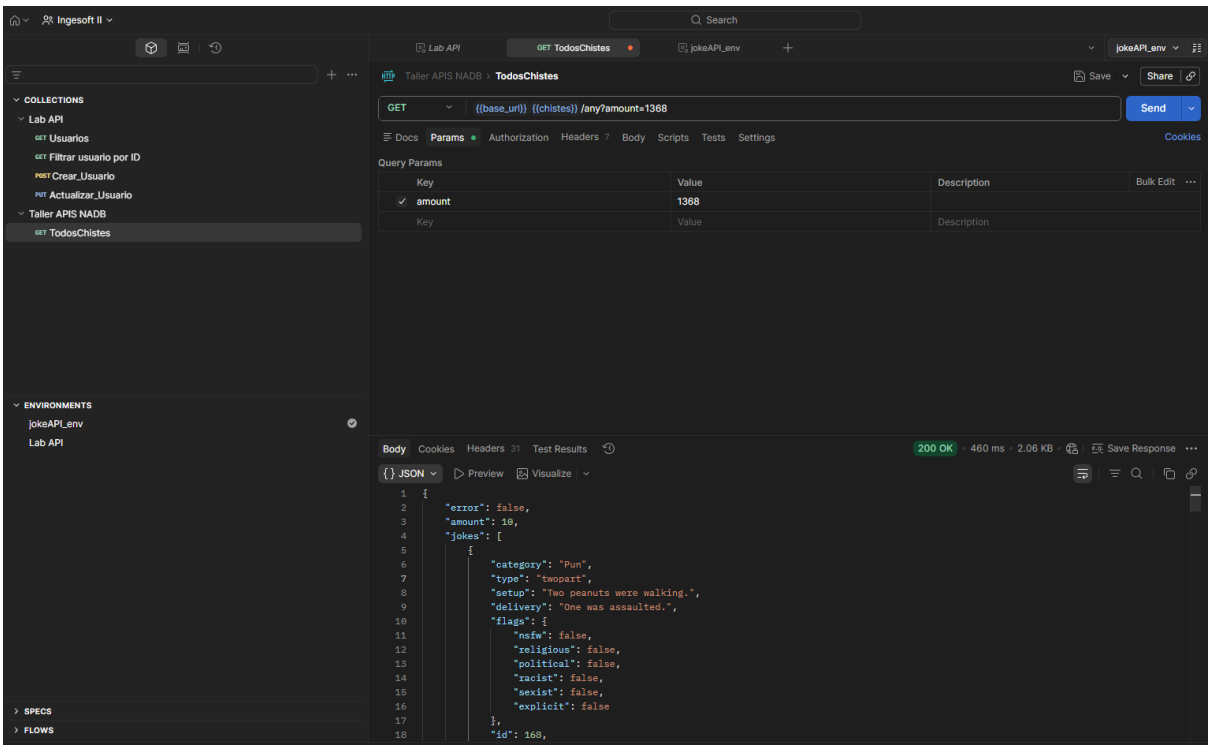
Todo esto disponible en resumen en markdown en su repositorio GitHub para revisar

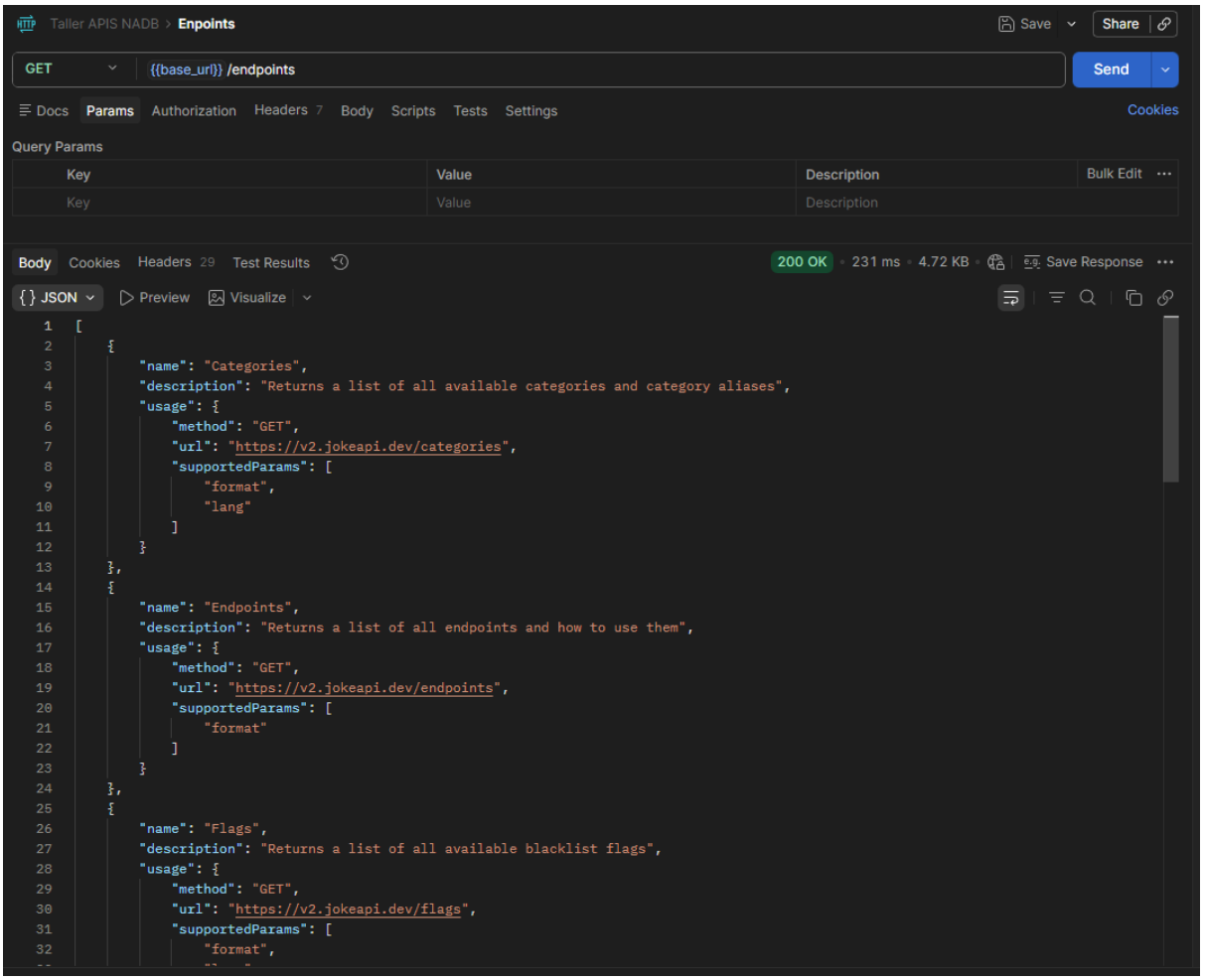
Solución:

1. **API ELEGIDA:** Joke API — Chistes
   https://v2.jokeapi.dev/joke

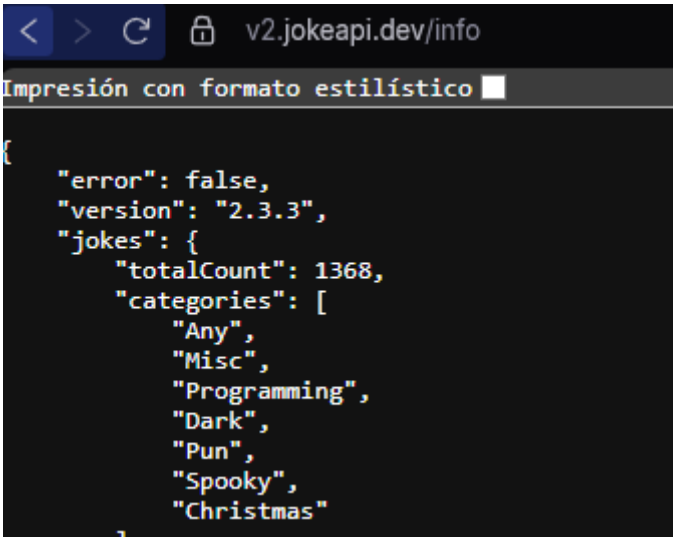
   GET /joke/Any → chiste aleatorio
   GET /joke/Programming → chiste de programación
   GET /joke/Any?blacklistFlags=nsfw → filtrar contenido
   Sin registro, sin token

2.
   - Esta API no tiene una estructura específica de petición para traer todas las bromas (por ejemplo) y tampoco se puede armar puesto que el número máximo de chistes en una petición es de 10, pero lo que sí podríamos hacer es ir a algún endpoint y traer toda la información específica de ese endpoint, en este caso voy al endpoint de los endpoints (valga la redundancia), además también voy a mostrar que está limitado a diez la cantidad de chistes

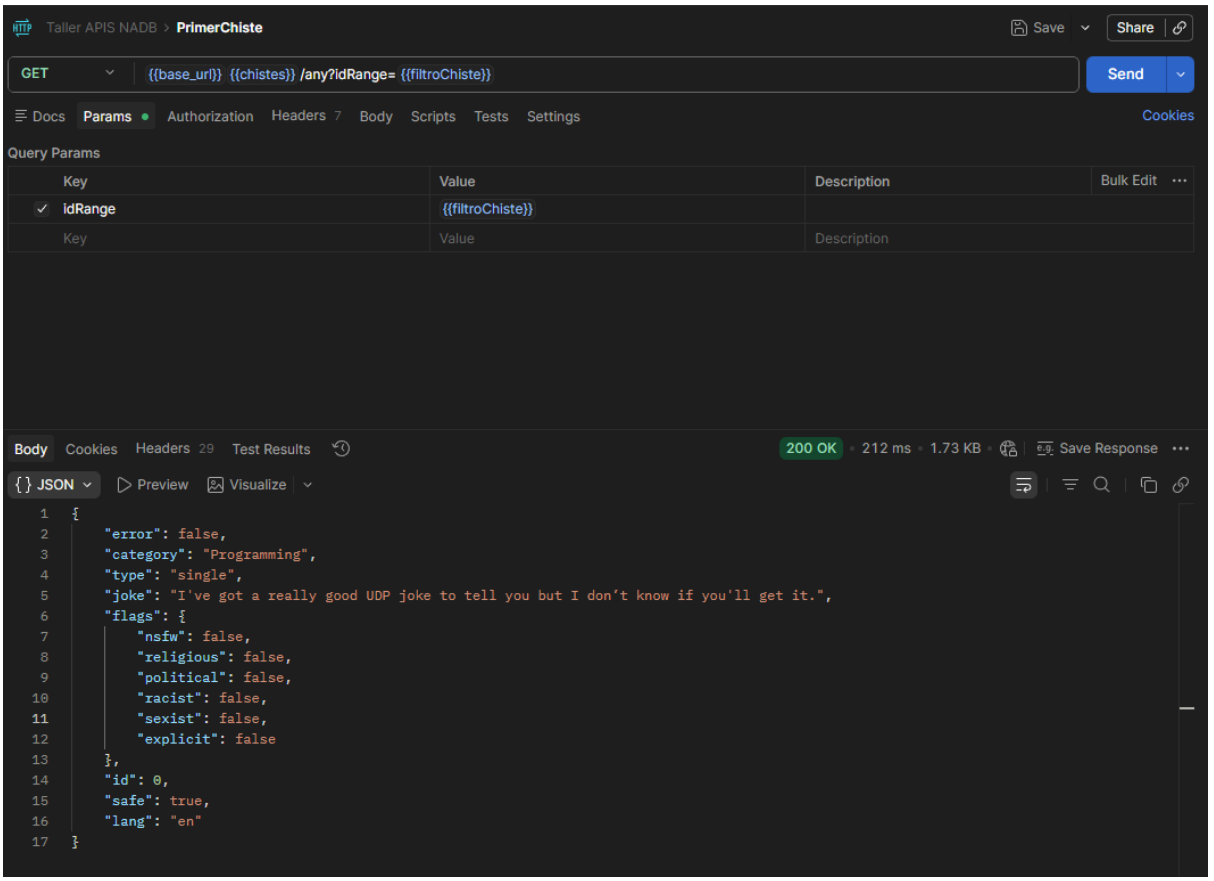




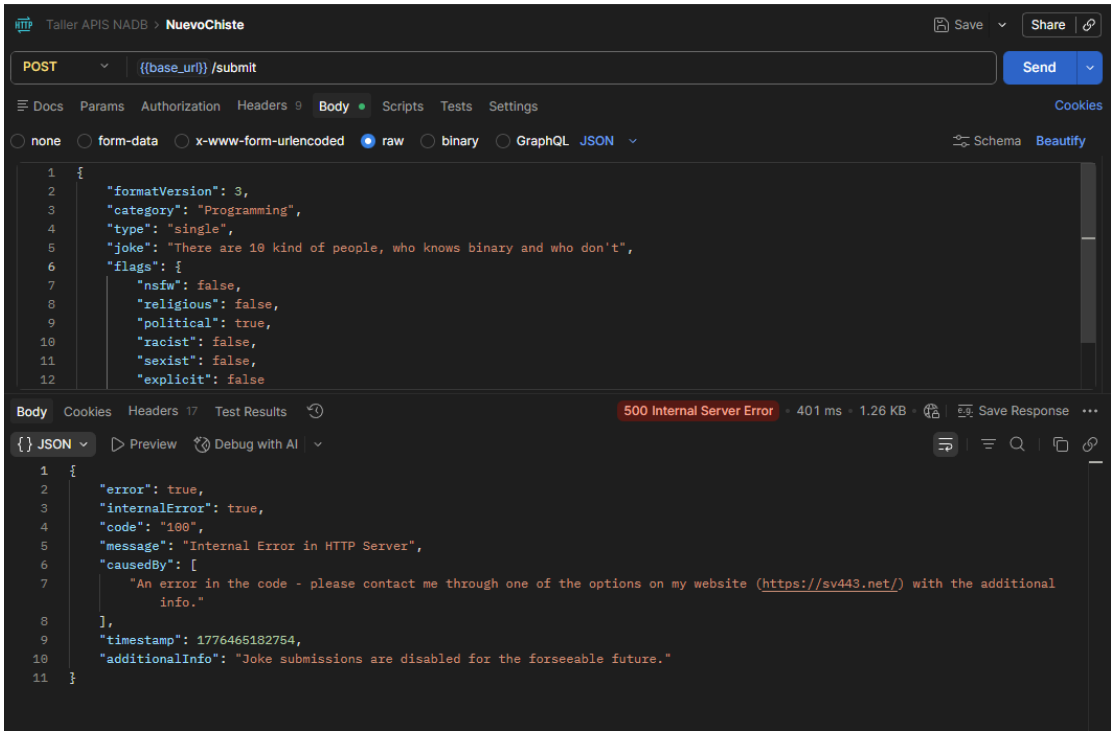
(Sé que son 1368 chistes porque en el endpoint de info se puede ver cuántos chistes existen en total)



- En este punto haré una petición para traer el primer chiste, para ello utilizaré la categoría any puesto que no sé de qué categoría es, y 0 como id pues el conteo de chistes empieza por el id 0.

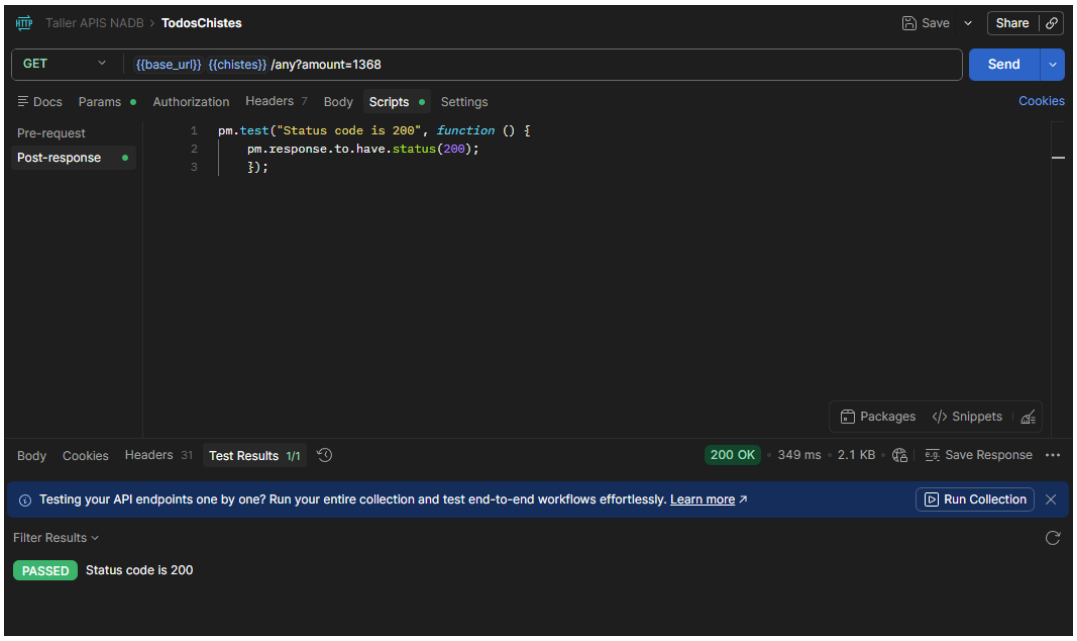


- Esta API tiene post para enviar un chiste tentativo que podría llegar a ser incluido, para ello existe el endpoint submit, este endpoint solo recibe datos en formato JSON por lo tanto el post lo haremos así:

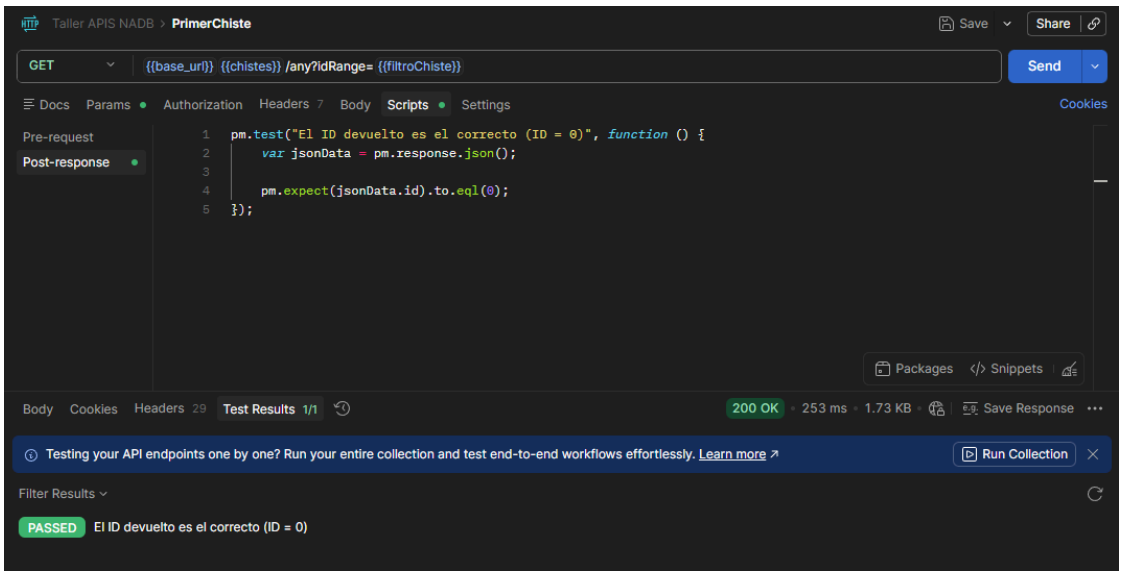


Lamentablemente está deshabilitado el envío de nuevos chistes.

- En la primera petición se utilizó un query param.
- Para hacer los test debemos generar scripts que verifiquen algo dentro de nuestra petición, lo primero que me recomendó postman y que se vio en clase fue el siguiente test para la primera petición:



Para el segundo test la idea es hacer algo un poco más elaborado, quiero por ejemplo que el id que pida en la petición del primer chiste sea verificado:



3. En el repositorio se encuentra el archivo .json
4.

   → ¿Qué API elegiste y por qué?

   Elegí el API de chistes porque la verdad es que estoy viejito y me pareció que podía llegar a dar gracia.

   → ¿Qué datos devuelve?

   Chistes.

   → ¿Usa token o no? ¿Qué tipo?

   No usa token, pero curiosamente este API ha sido atacado a través de DoS. Por ello han limitado las respuestas y peticiones por minuto.

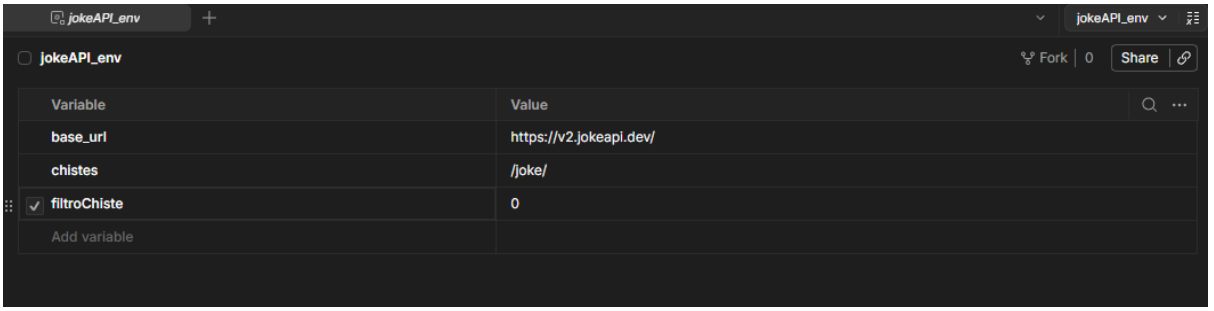
   → ¿Qué código de estado recibiste en cada request?

   En casi todos los casos fue recibido un código 200, en la única petición que no pasó fue en la de POST, pues fue deshabilitado por los creadores tal endpoint. en ella salió un código 500.

   → ¿Qué aprendiste diferente a JSONPlaceholder?

   La verdad me llamó la atención que en esta API no se pudieran obtener todos los chistes con un endpoint o parámetro específico.

Las variables de entorno utilizadas fueron las siguientes:

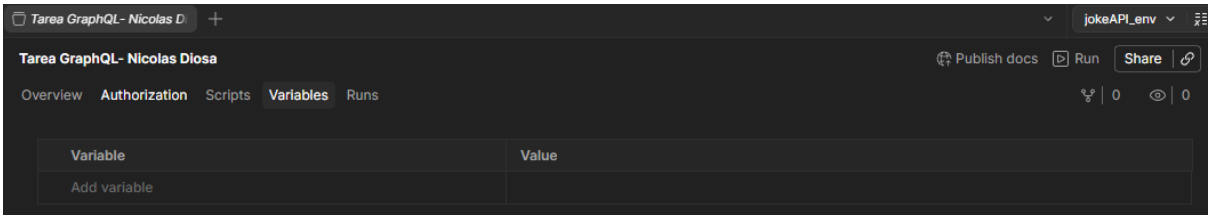


PARTE GRAPHQL:

1. Crear una colección en Postman llamada "Tarea GraphQL - [Nombre]"
2. Mínimo 5 requests con queries diferentes
3. Al menos una query anidada (datos relacionados)
4. Al menos una query con filtro por argumento
5. Mínimo 2 tests automáticos por request

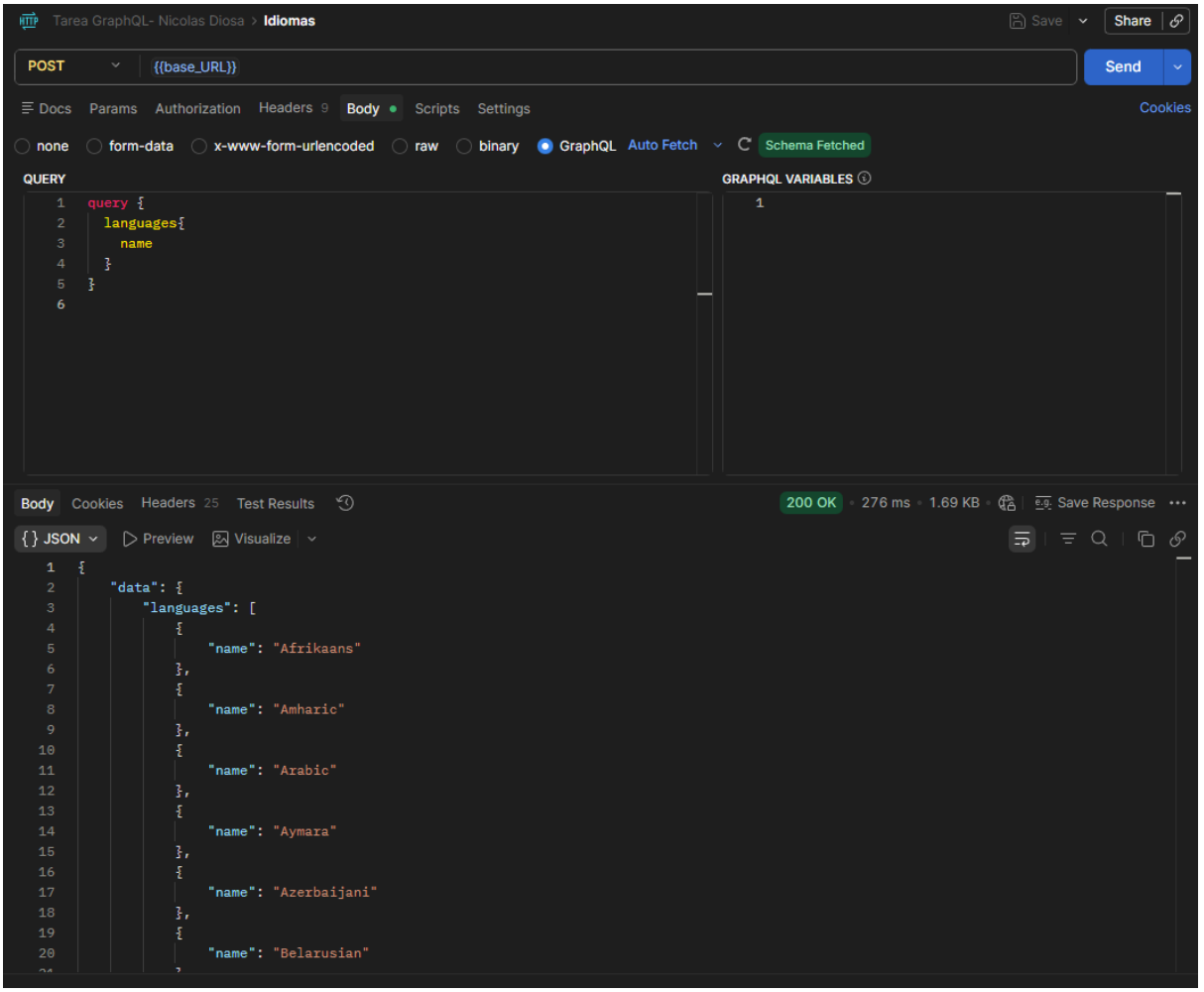
Solución:

1.

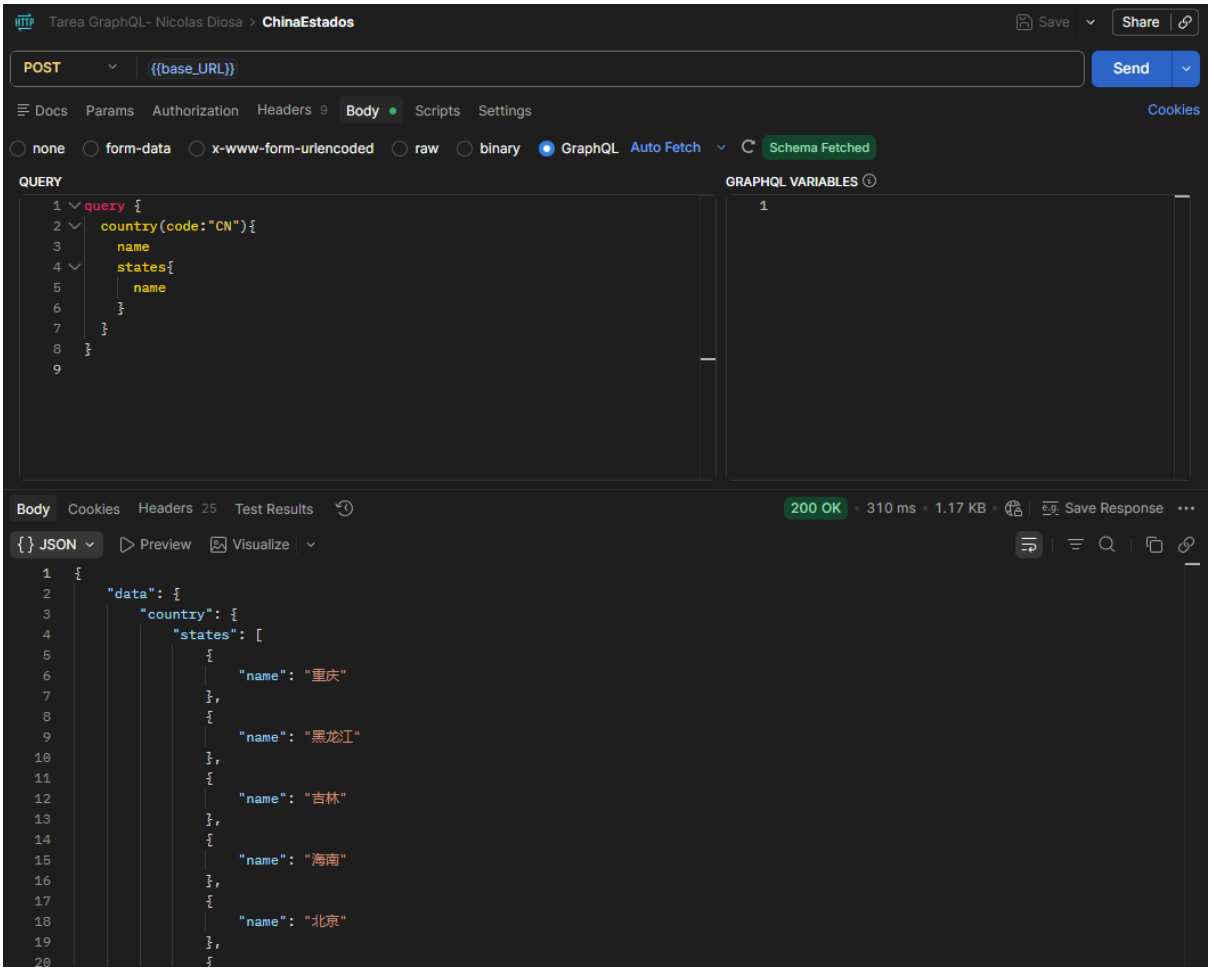


2.

2.1. Lo primero es traer todos los idiomas:



2.2. Los estados de China:



2.3. Monedas por continente

Tarea GraphQL- Nicolas Diosa > **MonedasContinentes**

Save | Share

POST {{base_URL}} Send

Docs | Params | Authorization | Headers 9 | Body | Scripts | Settings | Cookie

none | form-data | x-www-form-urlencoded | raw | binary | GraphQL | Auto Fetch | Schema Fetched

QUERY

```
query {
  continents{
    name
    countries{
      currency
    }
  }
}
```

GRAPHQL VARIABLES

```
1
```

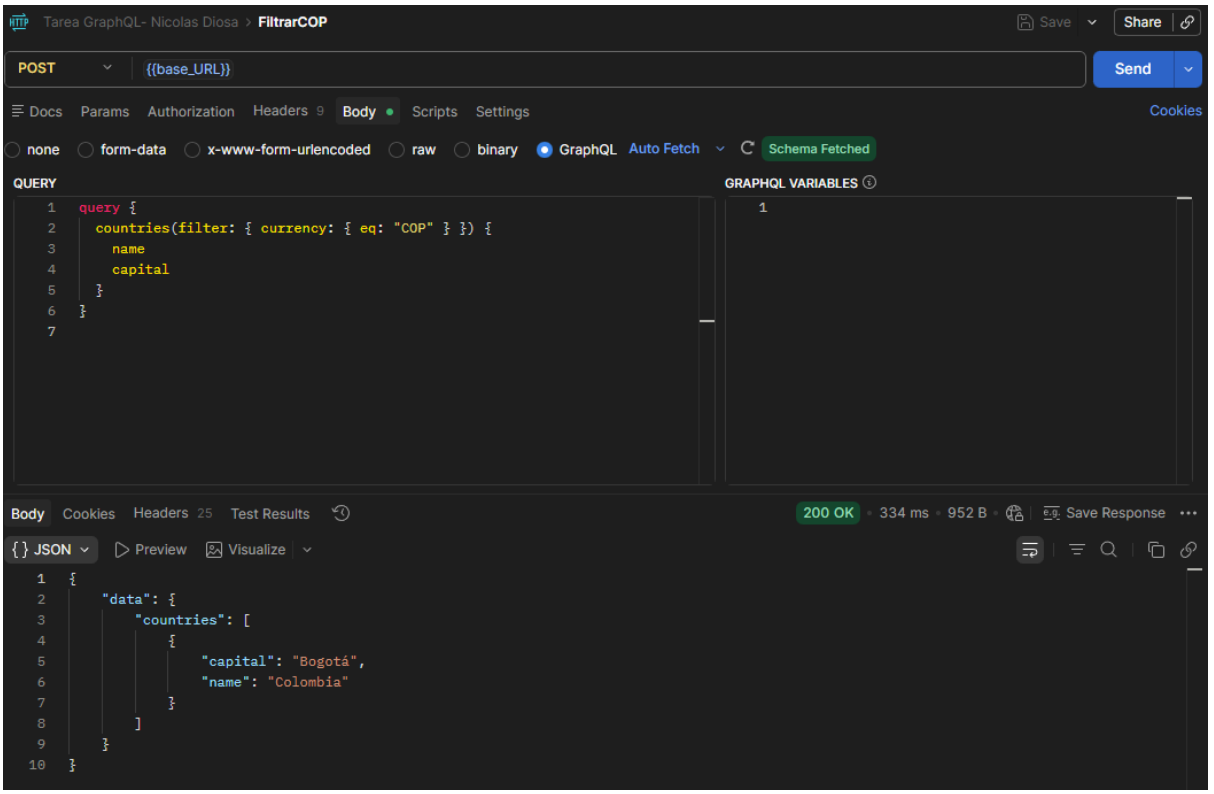
Body | Cookies | Headers 25 | Test Results

200 OK • 425 ms • 2.12 KB • Save Response

{} JSON | Preview | Visualize


```
{
    "data": {
        "continents": [
            {
                "name": "Africa",
                "countries": [
                    {
                        "currency": "AOA"
                    },
                    {
                        "currency": "XOF"
                    },
                    {
                        "currency": "BIF"
                    },
                    {
                        "currency": "XOF"
                    },
                    {
                        "currency": "BWP"
```

2.4. Filtrar al país por su moneda (Colombia)



2.5. Filtrar a colombia por su moneda y luego ver todos los países del mismo continente con su moneda:

Tarea GraphQL- Nicolas Diosa › **ConsultaAnidada**

Save | Share

POST {{base_URL}} Send

Docs | Params | Authorization | Headers 9 | Body | Scripts | Settings | Cookies

none | form-data | x-www-form-urlencoded | raw | binary | GraphQL | Auto Fetch | Schema Fetched

QUERY

```
query {
  countries(filter: { currency: { eq: "COP" } }) {
    name
    capital
    continent{
      countries{
        name
        currency
      }
    }
  }
}
```

GRAPHQL VARIABLES

```
1
```

Body | Cookies | Headers 25 | Test Results | 200 OK · 371 ms · 1.15 KB | Save Response

{} JSON | Preview | Visualize


```
{
    "data": {
        "countries": [
            {
                "capital": "Bogotá",
                "continent": {
                    "countries": [
                        {
                            "currency": "ARS",
                            "name": "Argentina"
                        },
                        {
                            "currency": "BOB,BOV",
                            "name": "Bolivia"
                        },
                        {
                            "currency": "BRL",
                            "name": "Brazil"
                        },
                        {
```


3. Los queries anidados pueden verse en el **2.2. 2.3. y 2.5.**
4. Los queries con filtros son **2.2. 2.4.** y **2.5.**

5.

Tarea GraphQL- Nicolas Diosa > **FiltrarCOP**

Save | Share

POST {{base_URL}} Send

Docs Params Authorization Headers 9 Body Scripts Settings Cookies

Pre-request
Post-response

```
pm.test("Bogotá es la capital de colombia", function () {
    var jsonData = pm.response.json();

    pm.expect(jsonData.data.countries[0].capital).to.eql("Bogotá");
});
pm.test("El  nomre del país es colombia", function () {
    var jsonData = pm.response.json();

    pm.expect(jsonData.data.countries[0].name).to.eql("Colombia");
});
```

Packages Snippets

Body Cookies Headers 25 Test Results 2/2 200 OK 197 ms 953 B Save Response

Testing your API endpoints one by one? Run your entire collection and test end-to-end workflows effortlessly. Learn more Run Collection

Filter Results

PASSED Bogotá es la capital de colombia

PASSED El nomre del país es colombia

Tarea GraphQL- Nicolas Diosa > **ConsultaAnidada**

Save | Share

POST {{base_URL}} Send

Docs Params Authorization Headers 9 Body Scripts Settings Cookies

Pre-request
Post-response

```
pm.test("El primer país de la lista es Colombia", function () {
    var jsonData = pm.response.json();

    pm.expect(jsonData.data.countries[0].name).to.eql("Colombia");
});
pm.test("Verificar que Argentina está en la lista de países", function () {
    var jsonData = pm.response.json();

    var nombresPaises = jsonData.data.countries[0].continent.countries.map(p => p.name);

    pm.expect(nombresPaises).to.include("Argentina");
});
```

Packages Snippets

Body Cookies Headers 25 Test Results 2/2 200 OK 260 ms 1.15 KB Save Response

Testing your API endpoints one by one? Run your entire collection and test end-to-end workflows effortlessly. Learn more Run Collection

Filter Results

PASSED El primer país de la lista es Colombia

PASSED Verificar que Argentina está en la lista de países

Tarea GraphQL- Nicolas Diosa > **Idiomas**

Save | Share

POST {{base_URL}} Send

Docs | Params | Authorization | Headers 9 | Body | Scripts | Settings | Cookies

Pre-request
Post-response

```
pm.test("El español está dentro de los idiomas", function () {
    var jsonData = pm.response.json();
    var nombresIdiomas = jsonData.data.languages.map(p => p.name);

   pm.expect(nombresIdiomas).to.include("Spanish");
});
pm.test("El inglés está dentro de los idiomas", function () {
    var jsonData = pm.response.json();
    var nombresIdiomas = jsonData.data.languages.map(p => p.name);

    pm.expect(nombresIdiomas).to.include("English");
});
```

Packages | Snippets

Body | Cookies | Headers 25 | Test Results 2/2 | 200 OK • 225 ms • 1.69 KB • Save Response

Testing your API endpoints one by one? Run your entire collection and test end-to-end workflows effortlessly. Learn more ↗ | Run Collection

Filter Results

PASSED El español está dentro de los idiomas

PASSED El inglés está dentro de los idiomas

Tarea GraphQL- Nicolas Diosa > **ChinaEstados**

Save

POST {{base_URL}}

Docs | Params | Authorization | Headers 9 | Body | Scripts | Settings

Pre-request
Post-response

```
pm.test("El estado  está dentro de china", function () {
    var jsonData = pm.response.json();
    var nombresEstados = jsonData.data.country.states.map(p => p.name);

   pm.expect(nombresEstados).to.include("重庆");
});
pm.test("El estado  está dentro de china", function () {
    var jsonData = pm.response.json();
    var nombresEstados = jsonData.data.country.states.map(p => p.name);

    pm.expect(nombresEstados).to.include("黑龙江");
});
```

Packages

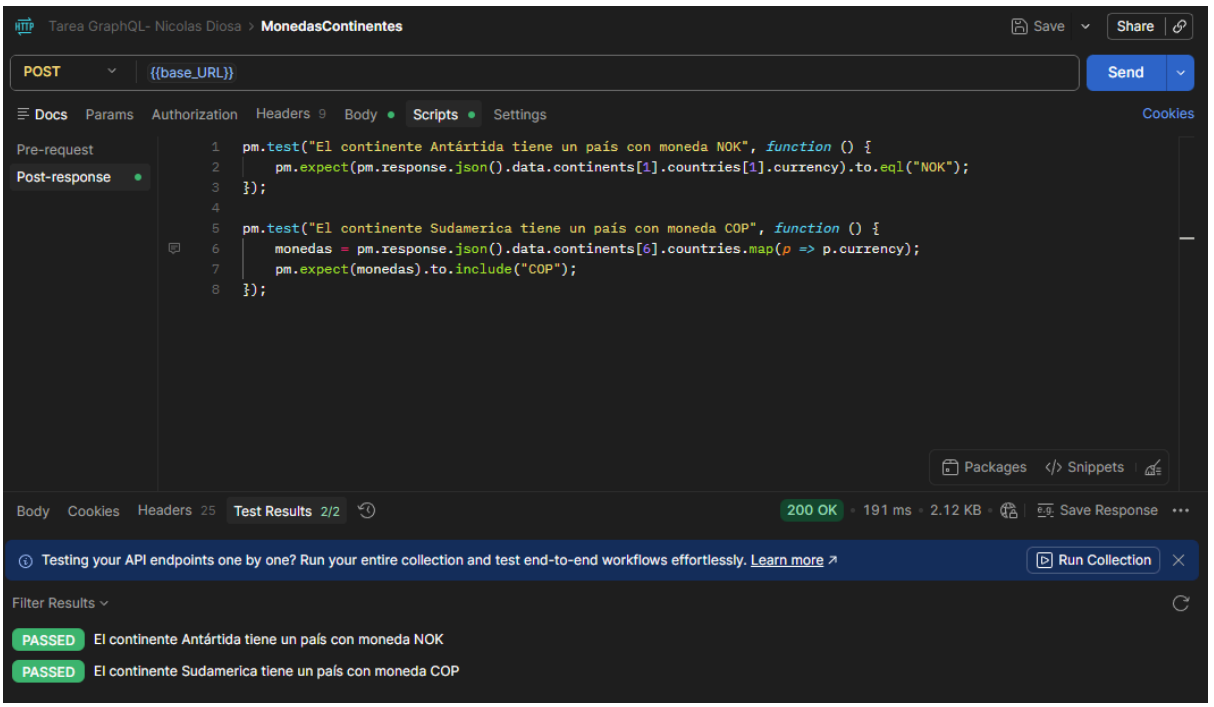
Body | Cookies | Headers 25 | Test Results 2/2 | 200 OK • 171 ms • 1.17 KB

Testing your API endpoints one by one? Run your entire collection and test end-to-end workflows effortlessly. Learn more ↗

Filter Results

PASSED El estado está dentro de china

PASSED El estado está dentro de china



→ ¿Qué diferencia encontraste vs REST?

La estructura de las peticiones se vuelve muy diferentes, se manejan casi como directorios, y también es posible anidar queries, cosa que en REST no parece tan fácil, evidente o posible de hacer.

→ ¿Cuántos requests REST necesitarías para reemplazar tu query más compleja?

En mis queries la más compleja de las estructuras es la de ConsultaAnidada, creo que necesitaría alrededor de 3 peticiones que luego extrapolaría para hallar el mismo resultado.

1. Encontrar a colombia por su moneda
2. Encontrar el continente en el que está Colombia
3. Traer a todos los países del continente.

→ ¿En qué proyecto real usarías GraphQL?

En proyectos que necesitaran traer información muy puntual del servidor, para evitar consultar información innecesaria o evitar traer menos información de la que necesite, además en proyectos en los que se necesiten usar API's con un límite de request, sería más viable utilizar Graphql que REST.